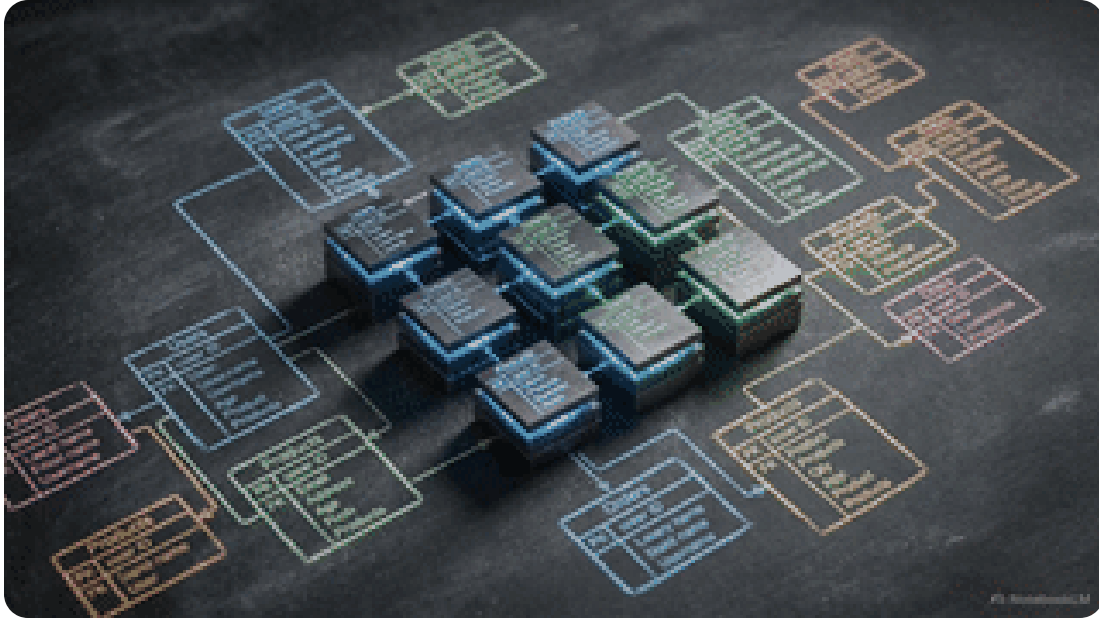




▶ Watch now

Here's the Secret to GenAI System Design Interviews

Stop buying \$100+ courses. Learn the framework that actually helps you think like an AI engineer.



AI ENGINEERING INSIDER

JUN 07, 2026

Most engineers prepare for system design interviews by memorizing architectures.

Recommendation system.

Search system.

News feed.

Ads ranking.

Fraud detection.

Chatbot.

RAG pipeline.

But GenAI system design interviews are different.

You are not only designing a model that predicts, ranks, or classifies.

You are designing a system that can generate.

↳ Text

↳ Images

↳ Audio

↳ Video

↳ Code

↳ Summaries

↳ Answers

↳ Agents

That changes the interview.

A GenAI system is not just a model endpoint.

It is a full product system with data pipelines, model architecture, training strategy, evaluation, safety, serving infrastructure, feedback loops, and monitoring.

The best candidates do not jump straight into “use GPT” or “use a Transformer.”

They show structured thinking.

Here is the framework I would use to answer almost any GenAI system design interview.

1. Start With Requirements

A weak candidate starts with the model.

A strong candidate starts with the problem.

Before talking about Transformers, diffusion models, RAG, vector databases, fine-tuning, or latency optimization, clarify the system requirements.

Ask:

- ↳ What is the business goal?
- ↳ Who are the users?
- ↳ What input does the system receive?
- ↳ What output should the system generate?
- ↳ Does it support text, image, audio, video, or multiple modalities?
- ↳ What quality level is expected?
- ↳ What latency is acceptable?
- ↳ Is real-time generation required?
- ↳ How many users will the system serve?
- ↳ What are the privacy, security, and safety constraints?

For example, “design a chatbot” is not specific enough.

A customer support chatbot is different from:

- ↳ A medical assistant
- ↳ A coding assistant
- ↳ A legal research assistant
- ↳ An enterprise knowledge assistant
- ↳ A personal AI tutor

Each one has different requirements, data sources, risks, latency targets, and evaluation criteria.

The same applies to image generation.

A fun consumer image generator is very different from a production-grade creative tool used by designers, advertisers, or e-commerce sellers.

Your first job in the interview is not to design immediately.

Your first job is to narrow the problem.

2. Frame the Problem as an ML Task

Once the requirements are clear, translate the product problem into an ML task.

Define the input and output.

Examples:

↳ Chatbot

Input: user text

Output: generated text

↳ Image captioning system

Input: image

Output: generated caption

↳ Text-to-image system

Input: text prompt

Output: generated image

↳ Video generation system

Input: text, image, or previous frames

Output: generated video

↳ Voice assistant

Input: audio

Output: text, speech, or action

This step matters because the input-output structure determines the model family.

If the output is a class, it may be a discriminative task.

If the output is new content, it is a generative task.

Then identify the task type:

↳ Classification

↳ Regression

↳ Text generation

↳ Image generation

↳ Audio generation

↳ Video generation

↳ Multimodal generation

This helps you choose the right modeling approach.

For example:

↳ Text generation usually points toward autoregressive Transformer models.

↳ Image generation often points toward diffusion models, GANs, VAEs, or Transformer-based image generators.

↳ Image captioning requires vision-language modeling.

↳ RAG-based question answering requires retrieval, ranking, grounding, and generation.

↳ Multimodal assistants may require separate encoders connected to a language model.

The goal is not to name-drop algorithms.

The goal is to explain why a particular modeling approach fits the problem.

3. Build the Data Strategy

GenAI systems are only as good as their data.

In traditional ML, data preparation often focuses on structured data:

- ↳ Tables
- ↳ Features
- ↳ Missing values
- ↳ Categorical encoding
- ↳ Feature stores
- ↳ Train-test splits

In GenAI, the focus shifts toward large-scale unstructured data:

- ↳ Text
- ↳ Images
- ↳ Videos
- ↳ Audio
- ↳ Documents
- ↳ Web pages
- ↳ User interactions
- ↳ Human feedback
- ↳ Synthetic data

A strong interview answer should explain the full data lifecycle.

Discuss:

- ↳ Where the data comes from
- ↳ How much data is needed

- ↳ Whether the data is labeled or unlabeled
- ↳ How the data is cleaned
- ↳ How duplicates are removed
- ↳ How harmful content is filtered
- ↳ How PII is detected and removed
- ↳ How bias is measured and mitigated
- ↳ How data is stored and retrieved efficiently
- ↳ How feedback data is collected after launch

For example, if you are designing an LLM-powered customer support system, the data may include:

- ↳ Product documentation
- ↳ Internal knowledge base articles
- ↳ Historical support tickets
- ↳ User manuals
- ↳ FAQs
- ↳ Chat transcripts
- ↳ Escalation examples
- ↳ Human-rated responses
- ↳ Safety policy examples

The data pipeline may include:

- ↳ Collection
- ↳ Parsing
- ↳ Cleaning
- ↳ Chunking
- ↳ Deduplication
- ↳ PII removal
- ↳ Toxicity filtering
- ↳ Embedding generation
- ↳ Indexing
- ↳ Quality scoring

This is where many candidates fail.

They talk about the model but ignore the data engine that powers it.

In GenAI system design, data is not a side topic.

Data is the foundation.

4. Choose the Model Architecture

Now you can talk about the model.

But the model choice should come after requirements, task framing, and data strategy.

Choose the architecture based on the system goal.

Examples:

↳ For a chatbot, use an autoregressive Transformer-based language model.

↳ For text-to-image generation, use a diffusion model with a text encoder.

↳ For image editing, use a conditional diffusion model.

↳ For image captioning, use a vision encoder connected to a language decoder.

↳ For enterprise question answering, use retrieval-augmented generation.

↳ For multimodal assistants, use vision/audio/video encoders connected to an LLM.

A strong answer includes trade-offs.

Do not just say, "Use an LLM."

Explain the decision.

For example:

- ↳ A larger model may improve reasoning quality, but increases latency and cost.
- ↳ A smaller model may be cheaper and faster, but less capable.
- ↳ Prompting is easy to iterate, but less controllable.
- ↳ Fine-tuning improves specialization, but requires high-quality labeled data.
- ↳ RAG improves factual grounding, but depends heavily on retrieval quality.
- ↳ Diffusion models generate high-quality images, but can be slow without optimization.
- ↳ Autoregressive models are flexible, but latency grows with output length.
- ↳ Multimodal models are powerful, but harder to train, evaluate, and debug.

The interviewer is not only testing whether you know model names.

They are testing judgment.

Can you choose the right architecture under real constraints?

That is the signal.

5. Explain Training and Optimization

Model training is not just “train the model on data.”

You need to explain the objective.

Different GenAI models are trained differently.

Examples:

- ↳ LLMs are commonly pretrained using next-token prediction.
- ↳ Diffusion models learn to denoise data incrementally.
- ↳ VAEs learn compressed latent representations and reconstruct data.
- ↳ GANs train a generator and a discriminator against each other.
- ↳ Vision-language models learn alignment between visual and textual representations.
- ↳ Instruction-tuned models learn from supervised examples of desired behavior.
- ↳ Aligned models may use preference optimization or human feedback.

A strong answer should cover:

- ↳ Training data
- ↳ Training objective
- ↳ Loss function
- ↳ Fine-tuning strategy
- ↳ Alignment strategy
- ↳ Evaluation dataset
- ↳ Hardware requirements
- ↳ Training cost
- ↳ Optimization techniques

Large-scale GenAI training is expensive, so mention efficiency techniques.

Examples:

- ↳ Mixed precision training
- ↳ Gradient checkpointing
- ↳ Distributed training
- ↳ Data parallelism
- ↳ Tensor parallelism
- ↳ Pipeline parallelism

- ↳ Quantization
- ↳ Distillation
- ↳ Parameter-efficient fine-tuning
- ↳ Caching
- ↳ Batch inference

You do not need to derive every formula in an interview.

But you should show that you understand the relationship between:

- ↳ Model size
- ↳ Data size
- ↳ Compute
- ↳ Cost
- ↳ Latency
- ↳ Quality
- ↳ Scalability

That is what makes the answer realistic.

6. Evaluate the System Properly

Evaluation is one of the hardest parts of GenAI system design.

Traditional ML metrics are often cleaner:

- ↳ Accuracy
- ↳ Precision
- ↳ Recall
- ↳ F1
- ↳ AUC
- ↳ RMSE
- ↳ MAE

GenAI outputs are open-ended.

There may be many valid answers.

A chatbot response can be fluent but wrong.

A summary can be concise but incomplete.

An image can be beautiful but fail to follow the prompt.

A code-generation model can produce code that appears correct but fails hidden tests.

So GenAI evaluation needs multiple layers.

For an LLM system, evaluate:

- ↳ Helpfulness
- ↳ Correctness
- ↳ Faithfulness
- ↳ Groundedness
- ↳ Hallucination rate
- ↳ Toxicity
- ↳ Bias
- ↳ Refusal quality
- ↳ Latency
- ↳ Cost per request
- ↳ User satisfaction
- ↳ Task completion rate

For an image generation system, evaluate:

- ↳ Prompt adherence
- ↳ Image quality
- ↳ Aesthetic quality
- ↳ Diversity

- ↳ Safety
- ↳ NSFW filtering
- ↳ User preference
- ↳ Generation latency

For a RAG system, evaluate:

- ↳ Retrieval precision
- ↳ Retrieval recall
- ↳ Context relevance
- ↳ Answer groundedness
- ↳ Citation accuracy
- ↳ Hallucination rate
- ↳ End-to-end answer quality

A strong evaluation strategy combines:

- ↳ Offline evaluation
- ↳ Human evaluation
- ↳ Automated evaluation
- ↳ LLM-as-a-judge
- ↳ Red-team testing
- ↳ Online A/B testing
- ↳ Production monitoring

But be careful with LLM-as-a-judge.

It is useful, but not perfect.

It can be biased, inconsistent, sensitive to prompt wording, and overly confident.

The best candidates acknowledge those limitations.

7. Design the End-to-End System

After discussing the model, zoom out.

A GenAI product is not just a model endpoint.

It is a complete production system.

For an LLM application, the architecture may include:

- ↳ User interface
- ↳ API gateway
- ↳ Authentication
- ↳ Rate limiting
- ↳ Prompt builder
- ↳ Input moderation
- ↳ Embedding model
- ↳ Vector database
- ↳ Retriever
- ↳ Reranker
- ↳ LLM inference service
- ↳ Post-processing layer
- ↳ Output moderation
- ↳ Logging pipeline
- ↳ Feedback collection
- ↳ Monitoring dashboard
- ↳ Human escalation path

For an image generation system, the architecture may include:

- ↳ Prompt processing
- ↳ Prompt safety filtering
- ↳ Text encoder
- ↳ Generation model
- ↳ Image safety classifier

- ↳ Watermarking
- ↳ Object storage
- ↳ CDN delivery
- ↳ User feedback loop
- ↳ Abuse detection
- ↳ Rate limiting

A strong system design answer connects ML decisions to production constraints.

Examples:

- ↳ If latency matters, use a smaller model, streaming, caching, quantization, or speculative decoding.
- ↳ If factuality matters, use RAG, citations, retrieval evaluation, and groundedness checks.
- ↳ If safety matters, use input moderation, output moderation, policy classifiers, red-teaming, and human review.
- ↳ If cost matters, use model routing, batch inference, caching, distillation, or smaller specialized models.
- ↳ If scalability matters, use load balancing, autoscaling, queueing, caching, and asynchronous processing.
- ↳ If reliability matters, use fallbacks, retries, circuit breakers, and model versioning.

The model is only one component.

The system around the model determines whether the product works in production.

8. Deploy, Monitor, and Improve

Deployment is not the finish line.

Deployment is the beginning of the feedback loop.

Once the GenAI system is live, monitor:

- ↳ Latency
- ↳ Error rate
- ↳ Throughput
- ↳ Token usage
- ↳ Cost per request
- ↳ Model drift
- ↳ Data drift
- ↳ Hallucination rate
- ↳ Safety violations
- ↳ User feedback
- ↳ Abuse patterns
- ↳ Retrieval failures
- ↳ Prompt injection attempts

You also need a continuous improvement loop.

That may include:

- ↳ Collecting user feedback
- ↳ Logging failed queries
- ↳ Building evaluation datasets
- ↳ Updating retrieval indexes
- ↳ Refreshing embeddings
- ↳ Fine-tuning on high-quality examples
- ↳ Running red-team tests
- ↳ Running A/B experiments
- ↳ Rolling out model versions safely
- ↳ Updating safety policies

A production GenAI system should version everything:

- ↳ Prompts
- ↳ Models
- ↳ Datasets
- ↳ Embeddings
- ↳ Retrieval logic
- ↳ Evaluation sets
- ↳ Safety filters
- ↳ Policy rules

Without versioning, debugging becomes painful.

Without monitoring, improvement becomes guesswork.

Without feedback loops, the system slowly becomes stale.

A Reusable GenAI System Design Template

Use this structure in interviews:

Step 1: Clarify Requirements

- ↳ Business goal
- ↳ Users
- ↳ Features
- ↳ Input and output
- ↳ Scale
- ↳ Latency
- ↳ Quality
- ↳ Privacy
- ↳ Safety

Step 2: Frame the ML Task

- ↳ Define input modality
- ↳ Define output modality
- ↳ Identify task type
- ↳ Choose a discriminative or a generative approach
- ↳ Explain why ML is needed

Step 3: Design the Data Pipeline

- ↳ Data sources
- ↳ Data collection
- ↳ Data cleaning
- ↳ Deduplication
- ↳ PII removal
- ↳ Bias mitigation
- ↳ Storage
- ↳ Indexing
- ↳ Feedback data

Step 4: Choose the Model

- ↳ Architecture
- ↳ Model size
- ↳ Modality support
- ↳ Fine-tuning strategy
- ↳ Trade-offs
- ↳ Cost and latency implications

Step 5: Train and Optimize

- ↳ Training objective
- ↳ Loss function
- ↳ Training data
- ↳ Fine-tuning
- ↳ Alignment

- ↳ Distributed training
- ↳ Quantization
- ↳ Distillation

Step 6: Evaluate

- ↳ Offline metrics
- ↳ Human evaluation
- ↳ Safety evaluation
- ↳ Bias testing
- ↳ Robustness testing
- ↳ Online A/B testing
- ↳ Production metrics

Step 7: Design the Production System

- ↳ APIs
- ↳ Model serving
- ↳ Retrieval
- ↳ Caching
- ↳ Safety filters
- ↳ Rate limiting
- ↳ Logging
- ↳ Monitoring
- ↳ Human escalation

Step 8: Monitor and Improve

- ↳ Track quality
- ↳ Track latency
- ↳ Track cost
- ↳ Track safety incidents
- ↳ Collect feedback
- ↳ Update data

- ↳ Improve prompts
 - ↳ Re-train or fine-tune
 - ↳ Roll out safely
-

Example Questions This Framework Applies To

This framework works for many GenAI system design interviews:

- ↳ Design ChatGPT
- ↳ Design an AI customer support agent
- ↳ Design a text-to-image generator
- ↳ Design an AI code assistant
- ↳ Design an email summarization system
- ↳ Design a multimodal search assistant
- ↳ Design an AI video generation platform
- ↳ Design an enterprise RAG system
- ↳ Design a personalized AI tutor
- ↳ Design an AI image editing tool
- ↳ Design an AI meeting assistant
- ↳ Design an AI research assistant

The exact architecture changes.

But the thinking process stays the same.

Final Thought

GenAI system design interviews are not about memorizing one perfect architecture.

They are about structured reasoning.

Can you clarify the problem?

Can you frame it as an ML task?

Can you design the data pipeline?

Can you justify the model architecture?

Can you explain training and optimization?

Can you evaluate open-ended outputs?

Can you design for safety, scale, latency, and cost?

Can you monitor and improve the system after launch?

That is what separates someone who knows GenAI concepts from someone who can design GenAI systems.

The model matters.

But the system around the model matters even more.